

TERMINAL

ls

List files and folders in the current directory

```
daffy@Angel-laptop MINGW64 /  
$ ls  
LICENSE.txt      cmd/  git-bash.exe*  proc/  unins000.exe*  
ReleaseNotes.html dev/  git-cmd.exe*   tmp/   unins000.msg  
bin/             etc/  mingw64/       unins000.dat  usr/
```

I can do it!

cd

Changes directory

```
daffy@Angel-laptop MINGW64 ~  
$ cd Downloads/
```

I can do it!

whoami

shows the current user

pwd

prints the current directory path

```
daffy@Angel-laptop MINGW64 ~/Downloads  
$ whoami  
daffy  
  
daffy@Angel-laptop MINGW64 ~/Downloads  
$ pwd  
/c/Users/daffy/Downloads
```

I can do it!

mkdir

Creates a new folder

```
daffy@Angel-laptop MINGW64 ~/Downloads/School  
$ mkdir test  
  
daffy@Angel-laptop MINGW64 ~/Downloads/School  
$ ls  
AzureTest/  
'Begreber i Objektorienteret Programmering i Java.pdf'  
'Domain modeling.jpg'  
Screenshot_2026-04-28_225455.png  
Screenshot_2026-04-28_225510.png  
Screenshot_2026-04-28_225517.png  
Screenshot_2026-04-28_225523.png  
Screenshot_2026-04-28_225539.png  
Screenshot_2026-04-28_225545.png  
Shneiderman.s.Eight.Golden.Rules.Worksheet.pdf  
"UTF-8 '2-Semester-Eksamensprojekt-Fora%CC%8Ar-2026.pdf"  
Videos/  
ideaIU-2025.2.exe*  
movie/  
spring-pokemon/  
test/
```

I can do it!

touch

Creates a new empty file

```
daffy@Angel-laptop MINGW64 ~/Downloads/School
$ cd test/

daffy@Angel-laptop MINGW64 ~/Downloads/School/test
$ touch test.txt

daffy@Angel-laptop MINGW64 ~/Downloads/School/test
$ cd

daffy@Angel-laptop MINGW64 ~
$ cd Downloads/School/test/

daffy@Angel-laptop MINGW64 ~/Downloads/School/test
$ ls
test.txt
```

I can do it!

cp

Copies files or folders

mv

Moves or renames a file

```
daffy@Angel-laptop MINGW64 ~/Downloads/School/test
$ cp test.txt copy.txt

daffy@Angel-laptop MINGW64 ~/Downloads/School/test
$ ls
copy.txt  test.txt

daffy@Angel-laptop MINGW64 ~/Downloads/School/test
$ mv copy.txt moved.txt

daffy@Angel-laptop MINGW64 ~/Downloads/School/test
$ ls
moved.txt  test.txt
```

I can do it!

rm

Deletes files or folders

nano

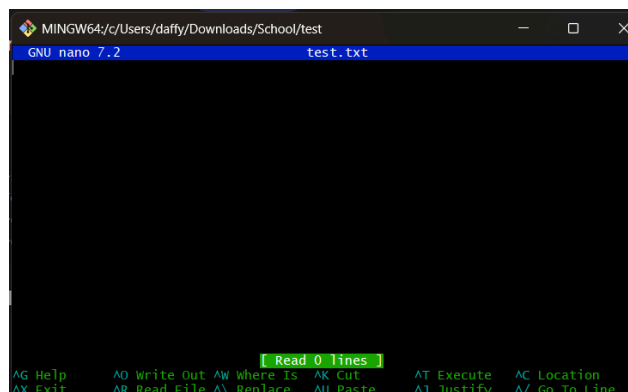
opens a text editor

(i think it's hard to use)

```
daffy@Angel-laptop MINGW64 ~/Downloads/School/test
$ rm moved.txt

daffy@Angel-laptop MINGW64 ~/Downloads/School/test
$ ls
test.txt

daffy@Angel-laptop MINGW64 ~/Downloads/School/test
$ nano test.txt
```



cat

displays file content

```
daffy@Angel-laptop MINGW64 ~/Downloads/School/test
$ nano test.txt

daffy@Angel-laptop MINGW64 ~/Downloads/School/test
$ cat test.txt
hello
```

I can do it!

wc

counts lines, words and characters

```
daffy@Angel-laptop MINGW64 ~/Downloads/School/test
$ wc test.txt
1 1 6 test.txt
```

I can do it!

sort

sorts lines in a file

uniq

removes duplicate lines

```
daffy@Angel-laptop MINGW64 ~/Downloads/School/test
$ nano test.txt

daffy@Angel-laptop MINGW64 ~/Downloads/School/test
$ sort test.txt
hello
hello2

daffy@Angel-laptop MINGW64 ~/Downloads/School/test
$ uniq test.txt
hello2
hello
```

I can do it!

GITHUB

git clone

Copies a repository from GitHub to your computer

```
daffy@Angel-laptop MINGW64 ~/Downloads/School/test
$ git clone https://github.com/Yuk1210/Angel-Mandatory-2.git
Cloning into 'Angel-Mandatory-2'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

I can do it!

git add

Stages changes for commit

git commit -m

Saves changes with a message

```
daffy@Angel-laptop MINGW64 ~/Downloads/School/test/Angel-Mandatory-2 (main)
$ ls
README.md

daffy@Angel-laptop MINGW64 ~/Downloads/School/test/Angel-Mandatory-2 (main)
$ touch file.txt
```

```
daffy@Angel-laptop MINGW64 ~/Downloads/School/test/Angel-Mandatory-2 (main)
$ git add .

daffy@Angel-laptop MINGW64 ~/Downloads/School/test/Angel-Mandatory-2 (main)
$ git commit -m "add file"
[main 924207c] add file
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 file.txt
```

I can do it!

git push

Uploads changes to GitHub

git pull

Downloads latest changes from GitHub

```
daffy@Angel-laptop MINGW64 ~/Downloads/School/test/Angel-Mandatory-2 (main)
$ git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 278 bytes | 278.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:Yuk1210/Angel-Mandatory-2.git
   ac56c51..924207c  main -> main

daffy@Angel-laptop MINGW64 ~/Downloads/School/test/Angel-Mandatory-2 (main)
$ git pull
Already up to date.
```

I can do it!

COMPUTER FUNDAMENTALS

hardware + Von Neumann

CPU: executes instructions

RAM: temporary memory

Storage: permanent data

Input/Output devices

Von Neumann: shared memory for data + instructions

I can do it!

how computers work

Hardware runs instructions

OS manages resources

Software runs on top of OS

I can do it!

processes

A process = running program

Managed by OS

Has memory + CPU time

I can do it!

number systems

Binary: base 2 (0,1) = machine level

Decimal: base 10 = human-readable

Hex: base 16 (0–9 + A–F) = memory addresses / color

I can do it!

charsets

ASCII: 128 characters

Unicode: supports all languages

Unicode uses more bytes

I can do it!

databases

Relational (MySQL) = structured data

NoSQL (MongoDB) = flexible/scalable

SQL

create database

Creates a new database

```
CREATE DATABASE testdb;
```

I can do it!

use

Selects a database to work in

```
USE testdb;
```

I can do it!

create table

Creates a new table

```
CREATE TABLE users (id INT, name VARCHAR(255));
```

I can do it!

insert

adds new data to a table

```
INSERT INTO users (id) VALUES (1);
```

I can do it!

select

Retrieves data from a table

```
SELECT * FROM users;
```

I can do it!

where

Filters results

```
SELECT * FROM users WHERE id = 1;
```

I can do it!

Primary Key

Unique identifier

Cannot be NULL

Foreign Key

Links tables together

I can do it!

limit

Limits number of results

```
SELECT * FROM users LIMIT 5;
```

I can do it!

order by

Sorts results

```
SELECT * FROM users ORDER BY name;
```

I can do it!

```
CREATE DATABASE testdb;

USE testdb;

CREATE TABLE users (id INT, name VARCHAR(255));
CREATE TABLE home (address INT);

INSERT INTO users (id) VALUES (1);
INSERT INTO users (name) VALUES ('Alice');
INSERT INTO users (name, id) VALUES ('bob', 2);

SELECT * FROM users;

SELECT * FROM users WHERE id = 1;

SELECT * FROM users LIMIT 5;

SELECT * FROM users ORDER BY name;
```

count

Counts rows

```
SELECT COUNT(*) FROM users;
```

I can do it!

like

Searches using patterns

```
SELECT * FROM users WHERE name LIKE 'A%';
```

I can do it!

join

Combines data from multiple tables

```
SELECT * FROM orders JOIN users ON orders.user_id = users.id;
```

I can do it!

update

Modifies existing data

```
UPDATE users SET name='bob' WHERE id=1;
```

I can do it!

delete

removes data

```
DELETE FROM users WHERE id=1;
```

I can do it!

start transaction

Begins a transaction

```
START TRANSACTION;
```

I can do it!

commit

Saves all changes in transaction

```
COMMIT;
```

I can do it!

rollback

Cancels changes in transaction

```
ROLLBACK;
```

I can do it!

```
SELECT COUNT(*) FROM users;

SELECT * FROM users WHERE name LIKE 'b%';

SELECT * FROM home JOIN users ON home.address = users.id;

UPDATE users SET name='Alice' WHERE id=3;

DELETE FROM users WHERE id=2;

START TRANSACTION;

COMMIT;

ROLLBACK;
```

GIT ADVANCED + YAML

pull request
Push branch
create PR on GitHub
I can do it!

git workflows
main
branch
PR
merge
I can do it!

merge conflict
Edit conflicting file
Remove markers
Commit again
I can do it!

can write yaml
name: Test
on: push
jobs:
build:
runs-on: ubuntu-latest
I can do it!

github actions
Understand structure
Runner - machine
Job - group of steps
Step - single task

Use cases
CI: Make sure your code doesn't break before merging.
CD: Website updates automatically
Testing
Deployment

cloud
IaaS - infrastructure
PaaS - platform
SaaS - software

AZURE

Push code
GitHub Actions
deploy to Azure App Service
I can do it!

flow:
Push
workflow runs
build
deploy

Add dependency
Configure application.properties
spring.datasource.url=jdbc:mysql://localhost:3306/testdb

database setup
Create DB
connect via connection string
I can do it!

problems
system crash
hardware failure
timeout

acid
Atomicity
Consistency
Isolation
Durability
Ensures safe transactions
I can do it!

transaction
Group queries together
Either all succeed or fail